

[Vai al contenuto](#)

DISPENSA DELLA LEZIONE

# Funzioni in JavaScript

Definizione, parametri, scope, regole di progettazione ed eventi sul DOM.



## CAPITOLO 1

# Che cos'è una funzione

**NOTA**

Una funzione è un blocco di codice con un nome. Viene eseguita **solo quando viene invocata**. Se non la chiami, il codice al suo interno non verrà mai eseguito.

Una funzione è un modo per raggruppare istruzioni sotto un nome, così da poterle richiamare ogni volta che servono senza riscriverle da capo. È il meccanismo base del riutilizzo del codice: scrivi la logica una volta sola, e la usi quante volte vuoi, in qualsiasi punto del programma.

Per definire una funzione si usa la keyword **function**, seguita dal nome scelto, dalle parentesi tonde **()** e dalle parentesi graffe **{ }** che racchiudono il corpo. Per eseguirla si scrive il nome seguito dalle parentesi tonde: questa operazione si chiama **invocazione**. Senza l'invocazione, il codice dentro la funzione non viene mai eseguito.

Il nome di una funzione deve essere **univoco**, non puoi avere due funzioni con lo stesso nome, non può contenere spazi, e non può coincidere con una parola che JavaScript riserva per sé. Per nomi composti da più parole si usa la convenzione **camelCase**.

## Esempio

```
// Definizione: il codice qui dentro NON viene ancora eseguito
function getAnnoCorrente() {
  console.log(2026);
}

// Invocazione: solo qui il codice viene eseguito
getAnnoCorrente(); // 2026
```

## CAPITOLO 2

# Il valore di ritorno: `return`

**NOTA**

`return` termina l'esecuzione della funzione e restituisce un valore al codice che l'ha invocata. Senza `return`, la funzione restituisce `undefined`.

Una funzione può non solo eseguire istruzioni, ma anche **produrre un valore** e restituirlo a chi la chiama. Per farlo si usa la keyword `return`. Quando JavaScript incontra `return`, interrompe immediatamente l'esecuzione della funzione e restituisce indietro il valore specificato. Quel valore può essere salvato in una variabile, usato in un calcolo, oppure passato a un'altra funzione.

Una funzione ben progettata si occupa **solo di elaborare una logica** e di restituire il risultato. Leggere dati dall'utente (`prompt`) o scrivere output (`console.log`, `alert`) non è compito suo: appartiene al codice esterno che usa la funzione. Questo approccio rende la funzione indipendente dal contesto e riutilizzabile ovunque: riceve valori tramite i parametri, li elabora e li restituisce con `return`. Il codice esterno decide poi cosa farne.

## Esempio

```
// La funzione si occupa solo del calcolo
function miaEta() {
    return 25;
}

function mioNome() {
    return "Marco";
}

// Il codice esterno decide cosa fare con i risultati
const nome = mioNome();
const eta = miaEta();

console.log('Età: ' + eta); // 25
console.log('Nome: ' + nome); // Marco
```

## CAPITOLO 3

# Parametri

**NOTA**

I **parametri** sono nomi segnaposto dichiarati nella definizione della funzione. Gli **argomenti** sono i valori reali passati durante l'invocazione.

Finora le funzioni che abbiamo scritto non ricevevano nulla dall'esterno. Per renderle davvero flessibili, possiamo dichiararle con uno o più parametri: nomi segnaposto scritti tra le parentesi tonde, separati da virgola. Nel momento in cui scrivi la funzione, i parametri non hanno ancora nessun valore, sono solo dei nomi riservati per i dati che arriveranno in seguito.

Il valore reale arriva quando la funzione viene invocata: i valori passati tra le parentesi tonde dell'invocazione si chiamano argomenti. All'atto dell'invocazione, ogni argomento viene associato al parametro corrispondente per posizione: il primo argomento va al primo parametro, il secondo al secondo, e così via. Dentro il corpo della funzione, i parametri si comportano esattamente come variabili locali. La stessa funzione, chiamata con argomenti diversi, produce risultati diversi: è questo che la rende riutilizzabile.

## Esempio

```
function saluta(nome) {  
    return 'Ciao, ' + nome + '!';  
}  
  
const messaggio1 = saluta('Mario'); // Ciao, Mario!  
const messaggio2 = saluta('Luigi'); // Ciao, Luigi!  
  
// 'prezzoNetto' e 'iva' sono parametri (segnaposto senza valore)  
function calcolaPrezzoLordo(prezzoNetto, iva) {  
    return prezzoNetto + (prezzoNetto * iva / 100);  
}  
  
// 100 e 22 sono argomenti: vanno rispettivamente a 'prezzoNetto' e 'iva'  
const lordo1 = calcolaPrezzoLordo(100, 22); // 122  
const lordo2 = calcolaPrezzoLordo(50, 10); // 55
```

## CAPITOLO 4

# Scope e visibilità

**NOTA**

Le variabili dichiarate *dentro* una funzione non sono visibili all'esterno. Le variabili dichiarate *fuori* sono visibili anche dentro le funzioni, ma dipendere da esse è una cattiva pratica.

Lo **scope** è la regola che determina da dove si può accedere ad una variabile. Una variabile dichiarata con **let** o **const** all'interno di una funzione esiste soltanto lì: nasce quando la funzione viene invocata e sparisce quando termina. Se provi ad accedervi dall'esterno, JavaScript lancia un errore di tipo **ReferenceError**.

Vale anche il contrario: una funzione **vede le variabili dichiarate fuori da essa**. Tecnicamente funziona, ma dipendere da variabili esterne rende la funzione fragile e difficile da riutilizzare, il suo comportamento cambia a seconda di cosa c'è scritto fuori, e non è più prevedibile. La pratica corretta è passare tutto ciò di cui la funzione ha bisogno tramite i parametri, in modo che sia completamente autosufficiente.

## Esempio

```
const aliquotaDefault = 22; // variabile esterna, visibile anche dentro le funzioni

function calcolaIvaDipendente(imponibile) {
  // Funziona, ma dipende da una variabile esterna: cattiva pratica
  return imponibile * aliquotaDefault / 100;
}

function calcolaIva(imponibile, aliquota) {
  // Riceve tutto tramite parametri: indipendente e riutilizzabile
  const risultato = imponibile * aliquota / 100;
  return risultato;
}

// 'risultato' non esiste qui fuori: è una variabile locale della funzione
// console.log(risultato); // ReferenceError
```

## CAPITOLO 5

# Le quattro regole d'oro

**NOTA**

Quattro principi guidano la progettazione di una buona funzione: **isolamento della logica**, **riutilizzabilità**, **indipendenza** e **atomicità**.

**Isolamento della logica.** La funzione gestisce solo l'elaborazione dei dati. Non legge input dall'utente e non scrive output sulla pagina o in console. Riceve dati tramite parametri, li elabora e restituisce un risultato con `return`. Il codice esterno si occupa di tutto il resto.

**Riutilizzabilità.** Una funzione ben scritta può essere invocata in punti diversi del programma, con valori diversi, senza essere modificata. Questo è possibile solo se riceve tutto ciò di cui ha bisogno tramite parametri, senza dipendere da variabili o contesti specifici.

**Indipendenza.** La funzione non legge né modifica variabili dichiarate fuori da essa. Dato lo stesso input, produce sempre lo stesso output, indipendentemente da cosa c'è intorno. Una funzione indipendente è prevedibile: il suo comportamento dipende solo dagli argomenti che riceve.

**Atomicità.** Ogni funzione deve fare *una cosa sola* e farla bene. Se una funzione calcola un valore, lo formatta e lo stampa, va suddivisa in tre funzioni separate. Funzioni piccole e focalizzate sono più semplici da leggere, da correggere e da combinare tra loro per costruire logiche più complesse.

## Esempio

```
// Tre funzioni atomiche, ognuna fa una cosa sola
function calcolaSubtotale(prezzo, quantita) {
    return prezzo * quantita;
}

function calcolaIva(imponibile, aliquota) {
    return imponibile * aliquota / 100;
}

function calcolaTotale(subtotale, iva) {
    return subtotale + iva;
}

// Il codice esterno combina le funzioni e gestisce l'output
const subtotale = calcolaSubtotale(30, 4);      // 120
const iva       = calcolaIva(subtotale, 22);    // 26.4
const totale    = calcolaTotale(subtotale, iva); // 146.4
console.log('Totale ordine: ' + totale + ' euro');
```

## CAPITOLO 6

# Eventi sui tag HTML

**NOTA**

Un **evento** è qualcosa che succede sulla pagina: un click, la pressione di un tasto, il passaggio del mouse su un elemento. È possibile reagire a questi eventi collegando una funzione direttamente a un tag HTML tramite attributi come `onclick`, `onkeypress`, `onmouseover`.

Il modo più diretto per far reagire un elemento della pagina a un'azione dell'utente è aggiungere un **attributo evento** direttamente sul tag HTML. Il nome di questi attributi inizia sempre con `on` seguito dal nome dell'evento — ad esempio `onclick` per il click, `onmouseover` quando il puntatore entra nell'elemento. Il valore dell'attributo è il codice JavaScript da eseguire: di solito l'invocazione di una funzione già definita nello script, completa di parentesi tonde.

**Esempio**

```
<button onclick="tiraDado()">Tira il dado</button>
<input type="text" onkeypress="contaCaratteri()">

```

```
function generaFaccia() {  
    return Math.floor(Math.random() * 6) + 1;  
}  
  
function tiraDado() {  
    const faccia = generaFaccia();  
    alert('Hai ottenuto: ' + faccia);  
}  
  
function contaCaratteri() {  
    // logica per contare i caratteri digitati  
}  
  
function ingrandisci() {  
    // logica per ingrandire l'immagine  
}  
  
function rimpicciolisci() {  
    // logica per ridurre l'immagine  
}
```

Gli attributi evento più comuni:

- **onclick** — click del mouse o tap su touch screen
- **ondblclick** — doppio click
- **onmouseover** — puntatore entra nell'area dell'elemento
- **onmouseout** — puntatore lascia l'area dell'elemento
- **onkeypress** — tasto premuto e rilasciato
- **onkeydown** — tasto premuto (si attiva prima di **onkeypress**)
- **onkeyup** — tasto rilasciato
- **oninput** — il valore di un campo di testo cambia
- **onchange** — il valore di un campo cambia e perde il focus
- **onsubmit** — un form viene inviato